



Yashaswi Education Society's

Reg No. Maha. : 417/2007/Pune

INTERNATIONAL INSTITUTE OF MANAGEMENT SCIENCE

An ISO 9001 Certified Institute

(Approved by AICTE Ministry of HRD Govt. of India, Recognised by Govt. of Maharashtra
and Affiliated to Savitribai Phule Pune University)

Campus. : IIMS Bldg, S. No. 169/1/A, Opp. Elpro International, Chinchwad, Pune - 411033. Ph.: (020) 27353730/32/33/34, Fax : (020) 27354731
Website. : www.iims.ac.in **E-mail :** info@iims.ac.in

Assignment No. 06

Title :- Programs for parsing of data, validations like Password, email, URL, etc

Date:-

Marks:-

Signature:-

Assignment No. 06

Problem Statement: Programs for parsing of data, validations like Password, email, URL, etc

Objective: - Learn the concept of Parsing of data

Theory :

Whenever the user accepts an input, it needs to be checked for validation which checks if the input data is what we are expecting. The validation can be done in two different ways, that is by using a flag variable or by using try or except which the flag variable will be set to false initially and if we can find out that the input data is what we are expecting the flag status can be set to true and find out what can be done next based on the status of the flag whereas while using try or except, a section of code is tried to run. If there is a negative response, then the except block of code is run.

Types of Validation in Python

There are three types of validation in python, they are:

- **Type Check:** This validation technique in python is used to check the given input data type. For example, int, float, etc.
- **Length Check:** This validation technique in python is used to check the given input string's length.
- **Range Check:** This validation technique in python is used to check if a given number falls in between the two numbers.

The syntax for validation in Python is given below:

Syntax using the flag:

```
flagName = False

while not flagName:

    if [Do check here]:

        flagName = True

    else:

        print('error message')
```

The status of the flag is set to false initially, and the same condition is considered for a while loop to make the statement while not true, and the validation is performed setting the flag to true if the validation condition is satisfied; otherwise, the error message is printed.

Syntax using an exception:

```
while True:

    try:

        [run code that might fail here]

    break

    except:

        print('This is the error message if the code fails')

print('run the code from here if code is successfully run in the try block of code above')
```

print('run the code from here if code is successfully run in the try block of code above')

Example

Let us see the following implementation to get better understanding

```
import re

def solve(s):
    pat = "^[a-zA-Z0-9- _]+@[a-zA-Z0-9]+\.[a-z]{1,3}$"
    if re.match(pat,s):
        return True
    return False

s = "popular_website15@comPany.com"
print(solve(s))
```

Input

```
"popular_website15@comPany.com"
```

Output

True

QUESTION:-

1. How does Python validate email and password?
2. How do I validate password and confirm password in Python?
3. How do you validate data in Python??
4. How do you validate an email address using regular expression in Python?
5. How do you validate a phone number in Python?
6. What is the best way to validate a telephone number in application?

CONCLUSION:- -----



Assignment No. 07

Title :- Programs for Pattern finding should be covered.

Date:-

Marks:-

Signature:-

Assignment No. 07

Problem Statement: Programs for Pattern finding should be covered..

Objective: - Learn & Understand the concept of Pattern finding

Theory :

In Python, for loop is used to print the various patterns. Printing the various patterns are most common asked programming questions in the interview. The multiple for loops are used to print the patterns where the first outer loop is used to print the number of rows, and the inner loop is used to print the number of columns. Most of the patterns use the following concepts.

- The outer loop to print the number of rows.
- The inner loops to print the number of columns.
- The variable to print whitespace according to the required place in Python

Patterns can be printed in python using simple for loops. **First outer loop** is used to handle the **number of rows** and the **Inner nested loop** is used to handle the **number of columns**. Manipulating the print statements, different number patterns, alphabet patterns, or star patterns can be printed. Some of the Patterns are shown in this article.

- **Simple pyramid pattern**

- Python3

```
# Python 3.x code to demonstrate star pattern
```

```
# Function to demonstrate printing pattern
```

```
def pypart(n):
```

```
    # outer loop to handle number of rows
```

```
    # n in this case
```

```
    for i in range(0, n):
```

```
        # inner loop to handle number of columns
```

```
        # values changing acc. to outer loop
```

```
        for j in range(0, i+1):
```

```
            # printing stars
```

```
            print("* ",end="")
```

```
        # ending line after each row
```

```
        print("\r")
```

Driver Code

n = 5

pypart(n)

Output

*

* *

* * *

* * * *

* * * * *

QUESTION:-

1. How do you make a python pattern program?

2. How do I read a python pattern program?

3 Which design pattern is used in Python?

4. What is the use of pattern in Python?

5.Design pattern

a.

*	*	1
**	**	2 3
***	***	4 5 6
****	****	7 8 9 10
*****	*****	11 12 13 14 15

A	A	*
B B	B C	* *
C C C	D E F	* * *
D D D D	G H I J	* * * *
E E E E E	K L M N O	* * * * *

CONCLUSION:- -----



Assignment No. 08

Title :- Programs covering all the aspects of Exception handling, user defined exception, Multithreading should be covered.

Date:-

Marks:-

Signature:-

Assignment No. 08

Problem Statement: Programs covering all the aspects of Exception handling, user defined exception, Multithreading should be covered.

Objective: - Learn & Understand the concept of Exception handling Multithreading

Theory :

Python throws errors and exceptions when there is a code gone wrong, which may cause the program to stop abruptly. Python also provides an exception handling method with the help of try-except. Some of the standard exceptions which are most frequent include IndexError, ImportError, IOError, ZeroDivisionError, TypeError, and FileNotFoundError.

User-Defined Exception in Python

Programmers may name their own exceptions by creating a new exception class. Exceptions need to be derived from the Exception class, either directly or indirectly. Although not mandatory, most of the exceptions are named as names that end in “**Error**” similar to the naming of the standard exceptions in python.

Deriving an Exception Class

Mostly names of the exception classes end with Error. We can define our own Exception Class in order to handle several distinct errors under a single module.

Python Code:

```
# NewError is a super class Exception
class NewError(Exception):
    pass

class ValueError(NewError):
    """When Value does not satisfy condition"""
    pass

Number = 5
while True:
    try:
```

```
num=int(input('Enter the number: '))  
if num < Number:  
    raise ValueError  
if num > Number:  
    raise ValueError  
break  
except ValueError:  
    print("Value does not matched")  
  
print("congratulations")
```

Creating a User-defined Exception class (Multiple Inheritance)

Derived class Exceptions are created when a single module handles multiple several distinct errors. Here we created a base class for exceptions defined by that module. This base class is inherited by various user-defined class to handle different types of errors.

Example

```
# define Python user-defined exceptions  
class Error(Exception):  
    """Base class for other exceptions"""  
    pass  
class Dividebyzero(Error):  
    """Raised when the input value is zero"""  
    pass  
try:  
    i_num = int(input("Enter a number: "))  
    if i_num ==0:
```

```
raise Dividebyzero
```

```
except Dividebyzero:
```

```
    print("Input value is zero, try again!")
```

```
    print()
```

Output

Enter a number: Input value is zero, try again!

There are several built-in exceptions in Python that are raised when errors occur. These built-in exceptions can be viewed using the **locals()** built-in functions as follows :

```
>>> locals()['__builtins__']
```

This returns a dictionary of built-in exceptions, functions and attributes.

Base Classes

The following exceptions are used mostly as base classes for other exceptions.

1. exception

BaseException

This is the base class for all built-in exceptions. It is not meant to be directly inherited by user-defined classes. For, user-defined classes, Exception is used. This class is responsible for creating a string representation of the exception using str() using the arguments passed. An empty string is returned if there are no arguments.

- **args** : The args are the tuple of arguments given to the exception constructor.
- **with_traceback(tb)** : This method is usually used in exception handling. This method sets tb as the new traceback for the exception and returns the exception object.

Code :

```
try:
    ...
except SomeException:
    tb = sys.exc_info()[2]
    raise OtherException(...).with_traceback(tb)
```

2. exception

Exception

This is the base class for all built-in non-system-exiting exceptions. All user-defined

exceptions should also be derived from this class.

3. exception

ArithmeticError

This class is the base class for those built-in exceptions that are raised for various arithmetic errors such as :

- OverflowError
- ZeroDivisionError
- FloatingPointError

Example :

try:

```
a = 10/0
```

```
print (a)
```

except ArithmeticError:

```
    print ("This statement is raising an arithmetic exception.")
```

else:

```
    print ("Success.")
```

Output :

This statement is raising an arithmetic exception.

4. exception

BufferError

This exception is raised when buffer related operations cannot be performed.

5. exception

LookupError

This is the base class for those exceptions that are raised when a key or index used on a mapping or sequence is invalid or not found. The exceptions raised are :

- KeyError
- IndexError

Example :

try:

```
a = [1, 2, 3]
```

```
print (a[3])
```

except LookupError:

```
print ("Index out of bound error.")
```

else:

```
print ("Success")
```

Output :

Index out of bound error.

Concrete exceptions

The following exceptions are the exceptions that are usually raised.

1. exception

AssertionError

An AssertionError is raised when an assert statement fails.

Example :

```
assert False, 'The assertion failed'
```

Output :

Traceback (most recent call last):

File "exceptions_AssertionError.py", line 12, in

```
assert False, 'The assertion failed'
```

AssertionError: The assertion failed

2. exception

AttributeError

An AttributeError is raised when an attribute reference or assignment fails such as when a non-existent attribute is referenced.

Example :

```
class Attributes(object):
```

```
    pass
```

```
object = Attributes()
```

```
print (object.attribute)
```

Output :

Traceback (most recent call last):

File "d912bae549a2b42953bc62da114ae7a7.py", line 5, in

print object.attribute

AttributeError: 'Attributes' object has no attribute 'attribute'

3. exception

EOFError

An EOFError is raised when built-in functions like input() hits an end-of-file condition (EOF) without reading any data. The file methods like readline() return an empty string when they hit EOF.

Example :

```
while True:
```

```
    data = input('Enter name : ')
```

```
    print ('Hello ', data)
```

Output :

```
Enter Name :Hello Aditi
```

```
Enter Name :Traceback (most recent call last):
```

```
File "exceptions_EOFError.py", line 13, in
```

```
    data = raw_input('Enter name :')
```

```
EOFError: EOF when reading a line
```

4. exception

FloatingPointError

A FloatingPointError is raised when a floating point operation fails. This exception is always defined, but can only be raised when Python is configured with the `--with-fpectl` option, or the `WANT_SIGFPE_HANDLER` symbol is defined in the `pyconfig.h` file.

Example :

```
import math
```

```
print (math.exp(1000))
```

Output :

Traceback (most recent call last):

File "", line 1, in

FloatingPointError: in math_1

5. exception

GeneratorExit

This exception directly inherits from BaseException instead of Exception since it is technically not an error. A GeneratorExit exception is raised when a generator or coroutine is closed.

Example :

```
def my_generator():  
    try:  
        for i in range(5):  
            print ('Yielding', i)  
            yield i  
    except GeneratorExit:  
        print ('Exiting early')
```

```
g = my_generator()  
print (g.next())  
g.close()
```

Output :

Yielding 0

0

Exiting early

6. exception

ImportError

An ImportError is raised when the import statement is unable to load a module or when the “from list” in from ... import has a name that cannot be found.

Example :

```
import module_does_not_exist
```

Output :

Traceback (most recent call last):

```
File "exceptions_ImportError_nomodule.py", line 12, in
import module_does_not_exist
```

ImportError: No module named module_does_not_exist

Example :

```
from exceptions import Userexception
```

Output :

Traceback (most recent call last):

```
File "exceptions_ImportError_missingname.py", line 12, in
from exceptions import Userexception
```

ImportError: cannot import name Userexception

7. exception

ModuleNotFoundError

This is the subclass of ImportError which is raised by import when a module could not be found. It is also raised when None is found in sys.modules.

8. exception

IndexError

An IndexError is raised when a sequence is referenced which is out of range.

Example :

```
array = [ 0, 1, 2 ]
print (array[3])
```

Output :

Traceback (most recent call last):

File "exceptions_IndexError.py", line 13, in

print array[3]

IndexError: list index out of range

9. exception

KeyError

A KeyError is raised when a mapping key is not found in the set of existing keys.

Example :

```
array = { 'a':1, 'b':2 }
```

```
print (array['c'])
```

Output :

Traceback (most recent call last):

File "exceptions_KeyError.py", line 13, in

```
print array['c']
```

KeyError: 'c'

10. exception

KeyboardInterrupt

This error is raised when the user hits the interrupt key such as Control-C or Delete.

Example :

```
try:
```

```
    print ('Press Return or Ctrl-C:')
```

```
    ignored = input()
```

```
except Exception, err:
```

```
    print ('Caught exception:', err)
```

```
except KeyboardInterrupt, err:
```

```
    print ('Caught KeyboardInterrupt')
```

```
else:
```

```
    print ('No exception')
```

Output :

Press Return or Ctrl-C: ^CCaught KeyboardInterrupt

11. exception

MemoryError

This error is raised when an operation runs out of memory.

Example :

```
def fact(a):  
    factors = []  
    for i in range(1, a+1):  
        if a%i == 0:  
            factors.append(i)  
    return factors
```

```
num = 600851475143  
print (fact(num))
```

Output :

Traceback (most recent call last):

```
File "4af5c316c749aff128df20714536b8f3.py", line 9, in  
    print fact(num)
```

```
File "4af5c316c749aff128df20714536b8f3.py", line 3, in fact  
    for i in range(1, a+1):
```

MemoryError

12. exception

NameError

This error is raised when a local or global name is not found. For example, an unqualified variable name.

Example :

```
def func():  
    print ans
```

```
func()
```

Output :

Traceback (most recent call last):

```
File "cfba0a5196b05397e0a23b1b5b8c7e19.py", line 4, in
```

```
func()
```

```
File "cfba0a5196b05397e0a23b1b5b8c7e19.py", line 2, in func
```

```
print ans
```

```
NameError: global name 'ans' is not defined
```

13. exception

NotImplementedError

This exception is derived from RuntimeError. Abstract methods in user defined classed should raise this exception when the derived classes override the method.

Example :

```
class BaseClass(object):
```

```
    """Defines the interface"""
```

```
    def __init__(self):
```

```
        super(BaseClass, self).__init__()
```

```
    def do_something(self):
```

```
        """The interface, not implemented"""
```

```
        raise NotImplementedError(self.__class__.__name__ + '.do_something')
```

```
class SubClass(BaseClass):
```

```
    """Implements the interface"""
```

```
    def do_something(self):
```

```
        """really does something"""
```

```
        print (self.__class__.__name__ + ' doing something!')
```

```
SubClass().do_something()
```

```
BaseClass().do_something()
```

Output :

Traceback (most recent call last):

```
File "b32fc445850cbc23cd2f081ba1c1d60b.py", line 16, in
```

```
    BaseClass().do_something()
```

```
File "b32fc445850cbc23cd2f081ba1c1d60b.py", line 7, in do_something
```

```
    raise NotImplementedError(self.__class__.__name__ + '.do_something')
```

NotImplementedError: BaseClass.do_something

14. exception

OSError([arg])

The OSError exception is raised when a system function returns a system-related error, including I/O failures such as “file not found” or “disk full” errors.

Example :

```
def func():  
    print (ans)
```

```
func()
```

Output :

Traceback (most recent call last):

```
File "442eccd7535a2704adbe372cb731fc0f.py", line 4, in  
    print i, os.ttyname(i)
```

OSError: [Errno 25] Inappropriate ioctl for device

15. exception

OverflowError

The OverflowError is raised when the result of an arithmetic operation is out of range. Integers raise MemoryError instead of OverflowError. OverflowError is sometimes raised for integers that are outside a required range. Floating point operations are not checked because of the lack of standardization of floating point exception handling in C.

Example :

```
import sys

print ('Regular integer: (maxint=%s)' % sys.maxint)

try:
    i = sys.maxint * 3
    print ('No overflow for ', type(i), 'i =', i)
except OverflowError, err:
    print ('Overflowed at ', i, err)

print()

print ('Long integer:')
for i in range(0, 100, 10):
    print ('%2d' % i, 2L ** i)

print()

print ('Floating point values:')
try:
    f = 2.0**i
    for i in range(100):
        print (i, f)
        f = f ** 2
except OverflowError, err:
    print ('Overflowed after ', f, err)
```

Output :

Regular integer: (maxint=9223372036854775807)

No overflow for i = 27670116110564327421

Long integer:

0 1

10 1024

20 1048576
30 1073741824
40 1099511627776
50 1125899906842624
60 1152921504606846976
70 1180591620717411303424
80 1208925819614629174706176
90 1237940039285380274899124224

Floating point values:

0 1.23794003929e+27
1 1.53249554087e+54
2 2.34854258277e+108
3 5.5156522631e+216

Overflowed after 5.5156522631e+216 (34, 'Numerical result out of range')

16. exception

RecursionError

The RecursionError is derived from the RuntimeError. This exception is raised when the interpreter detects that the maximum recursion depth is exceeded.

17. exception

ReferenceError

The ReferenceError is raised when a weak reference proxy is used to access an attribute of the referent after the garbage collection.

Example :

```
import gc
import weakref

class Foo(object):

    def __init__(self, name):
        self.name = name

    def __del__(self):
        print('(Deleting %s)' % self)

obj = Foo('obj')
p = weakref.proxy(obj)

print('BEFORE:', p.name)
obj = None
print('AFTER:', p.name)
```

Output :

```
BEFORE: obj
(Deleting )
AFTER:
```

Traceback (most recent call last):

```
File "49d0c29d8fe607b862c02f4e1cb6c756.py", line 17, in
    print 'AFTER:', p.name
```

ReferenceError: weakly-referenced object no longer exists

18. exception

RuntimeError

The RuntimeError is raised when no other exception applies. It returns a string indicating what precisely went wrong.

19. exception

StopIteration

The StopIteration error is raised by built-in function next() and an iterator's __next__() method to signal that all items are produced by the iterator.

Example :

```
Arr = [3, 1, 2]
```

```
i=iter(Arr)
```

```
print (i)
```

```
print (i.next())
```

```
print (i.next())
```

```
print (i.next())
```

```
print (i.next())
```

Output :

```
3
```

```
1
```

```
2
```

Traceback (most recent call last):

File "2136fa9a620e14f8436bb60d5395cc5b.py", line 8, in

```
print i.next()
```

StopIteration

20. exception

SyntaxError

The SyntaxError is raised when the parser encounters a syntax error. A syntax error may occur in an import statement or while calling the built-in functions exec() or eval(), or when reading the initial script or standard input.

Example :

try:

```
print (eval('geeks for geeks'))  
except SyntaxError, err:  
    print ('Syntax error %s (%s-%s): %s' % \  
          (err.filename, err.lineno, err.offset, err.text))  
print (err)
```

Output :

```
Syntax error (1-9): geeks for geeks  
invalid syntax (, line 1)
```

21. exception

SystemError

The SystemError is raised when the interpreter finds an internal error. The associated value is a string indicating what went wrong.

22. exception

SystemExit

The SystemExit is raised when sys.exit() function is called. A call to sys.exit() is translated into an exception to execute clean-up handlers (finally clauses of try statements) and to debug a script without running the risk of losing control.

23. exception

TypeError

TypeError is raised when an operation or function is applied to an object of inappropriate type. This exception returns a string giving details about the type mismatch.

Example :

```
arr = ('tuple', ) + 'string'  
print (arr)
```

Output :

Traceback (most recent call last):

File "30238c120c0868eba7e13a06c0b1b1e4.py", line 1, in

```
arr = ('tuple', ) + 'string'
```

TypeError: can only concatenate tuple (not "str") to tuple

24. exception

UnboundLocalError

UnboundLocalError is a subclass of NameError which is raised when a reference is made to a local variable in a function or method, but no value has been assigned to that variable.

Example :

```
def global_name_error():
    print (unknown_global_name)

def unbound_local():
    local_val = local_val + 1
    print (local_val)

try:
    global_name_error()
except NameError, err:
    print ('Global name error:', err)

try:
    unbound_local()
except UnboundLocalError, err:
    print ('Local name error:', err)
```

Output :

Global name error: global name 'unknown_global_name' is not defined
Local name error: local variable 'local_val' referenced before assignment

25. exception

UnicodeError

This exception is a subclass of ValueError. UnicodeError is raised when a Unicode-related encoding or decoding error occurs.

26. exception

ValueError

A ValueError is raised when a built-in operation or function receives an argument that has the right type but an invalid value.

Example :

```
print(int('a'))
```

Output :

Traceback (most recent call last):

File "44f00efda935715a3c5468d899080381.py", line 1, in

```
print(int('a'))
```

ValueError: invalid literal for int() with base 10: 'a'

27. exception

ZeroDivisionError

A ZeroDivisionError is raised when the second argument of a division or modulo operation is zero. This exception returns a string indicating the type of the operands and the operation.

Example :

```
print(1/0)
```

Output :

Traceback (most recent call last):

File "c31d9626b41e53d170a78eac7d98cb85.py", line 1, in

```
print 1/0
```

ZeroDivisionError: integer division or modulo by zero

QUESTION:-

1. How does Python handle user-defined exceptions?
2. Can we define user-defined exceptions in Python??
- 3 What are the 3 major exception types in Python?
4. What are the exceptions?
- 5.



Yashaswi Education Society's

Reg No. Maha. : 417/2007/Pune

INTERNATIONAL INSTITUTE OF MANAGEMENT SCIENCE

An ISO 9001 Certified Institute

(Approved by AICTE Ministry of HRD Govt. of India, Recognised by Govt. of Maharashtra
and Affiliated to Savitribai Phule Pune University)

Campus. : IIMS Bldg, S. No. 169/1/A, Opp. Elpro International, Chinchwad, Pune - 411033. Ph.: (020) 27353730/32/33/34, Fax : (020) 27354731
Website. : www.iims.ac.in E-mail : info@iims.ac.in

CONCLUSION:- -----



Assignment No. 09

Title :- Programs for Pattern finding should be covered.

Date:-

Marks:-

Signature:-

Assignment No. 09

Problem Statement: Programs demonstrating the IO operations like reading from file, writing into file from different file types like data file, binary file, etc...

Objective: - Learn & Understand the concept of Pattern finding

Theory :

Files

Files are named locations on disk to store related information. They are used to permanently store data in a non-volatile memory (e.g. hard disk).

Since Random Access Memory (RAM) is volatile (which loses its data when the computer is turned off), we use files for future use of the data by permanently storing them.

When we want to read from or write to a file, we need to open it first. When we are done, it needs to be closed so that the resources that are tied with the file are freed.

Hence, in Python, a file operation takes place in the following order:

1. Open a file
2. Read or write (perform operation)
3. Close the file

Opening Files in Python

Python has a built-in `open()` function to open a file. This function returns a file object, also called a handle, as it is used to read or modify the file accordingly.

```
>>> f = open("test.txt") # open file in current directory
>>> f = open("C:/Python38/README.txt") # specifying full path
```

We can specify the mode while opening a file. In mode, we specify whether we want to read `r`, write `w` or append `a` to the file. We can also specify if we want to open the file in text mode or binary mode.

The default is reading in text mode. In this mode, we get strings when reading from the file.

On the other hand, binary mode returns bytes and this is the mode to be used when dealing with non-text files like images or executable files.

Mode	Description
<code>r</code>	Opens a file for reading. (default)
<code>w</code>	Opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
<code>x</code>	Opens a file for exclusive creation. If the file already exists, the operation fails.
<code>a</code>	Opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
<code>t</code>	Opens in text mode. (default)
<code>b</code>	Opens in binary mode.
<code>+</code>	Opens a file for updating (reading and writing)

```
f = open("test.txt")    # equivalent to 'r' or 'rt'
f = open("test.txt",'w') # write in text mode
f = open("img.bmp",'r+b') # read and write in binary mode
```

Unlike other languages, the character `a` does not imply the number 97 until it is encoded using `ASCII` (or other equivalent encodings).

Moreover, the default encoding is platform dependent. In windows, it is `cp1252` but `utf-8` in

Linux.

So, we must not also rely on the default encoding or else our code will behave differently in different platforms.

Hence, when working with files in text mode, it is highly recommended to specify the encoding type.

```
f = open("test.txt", mode='r', encoding='utf-8')
```

Closing Files in Python

When we are done with performing operations on the file, we need to properly close the file.

Closing a file will free up the resources that were tied with the file. It is done using the `close()` method available in Python.

Python has a garbage collector to clean up unreferenced objects but we must not rely on it to close the file.

```
f = open("test.txt", encoding = 'utf-8')  
# perform file operations  
f.close()
```

This method is not entirely safe. If an exception occurs when we are performing some operation with the file, the code exits without closing the file.

A safer way is to use a [try...finally](#) block.

```
try:  
    f = open("test.txt", encoding = 'utf-8')  
    # perform file operations  
finally:
```

`f.close()`

This way, we are guaranteeing that the file is properly closed even if an exception is raised that causes program flow to stop.

The best way to close a file is by using the `with` statement. This ensures that the file is closed when the block inside the `with` statement is exited.

We don't need to explicitly call the `close()` method. It is done internally.

```
with open("test.txt", encoding = 'utf-8') as f:  
    # perform file operations
```

Writing to Files in Python

In order to write into a file in Python, we need to open it in write `w`, append `a` or exclusive creation `x` mode.

We need to be careful with the `w` mode, as it will overwrite into the file if it already exists. Due to this, all the previous data are erased.

Writing a string or sequence of bytes (for binary files) is done using the `write()` method. This method returns the number of characters written to the file.

```
with open("test.txt", 'w', encoding = 'utf-8') as f:  
    f.write("my first file\n")  
    f.write("This file\n\n")  
    f.write("contains three lines\n")
```

This program will create a new file named `test.txt` in the current directory if it does not exist. If it does exist, it is overwritten.

We must include the newline characters ourselves to distinguish the different lines.

Reading Files in Python

To read a file in Python, we must open the file in reading `r` mode.

There are various methods available for this purpose. We can use the `read(size)` method to read in the `size` number of data. If the `size` parameter is not specified, it reads and returns up to the end of the file.

We can read the `test.txt` file we wrote in the above section in the following way:

```
>>> f = open("test.txt",'r',encoding = 'utf-8')
>>> f.read(4) # read the first 4 data
'This'

>>> f.read(4) # read the next 4 data
'is '

>>> f.read() # read in the rest till end of file
'my first file\nThis file\ncontains three lines\n'

>>> f.read() # further reading returns empty sting
''
```

We can see that the `read()` method returns a newline as `\n`. Once the end of the file is reached, we get an empty string on further reading.

We can change our current file cursor (position) using the `seek()` method. Similarly, the `tell()` method returns our current position (in number of bytes).

```
>>> f.tell() # get the current file position
56

>>> f.seek(0) # bring file cursor to initial position
0

>>> print(f.read()) # read the entire file
This is my first file
```

contains three lines

We can read a file line-by-line using a [for loop](#). This is both efficient and fast.

```
>>> for line in f:
...     print(line, end = "")
...
This is my first file
This file
contains three lines
```

In this program, the lines in the file itself include a newline character `\n`. So, we use the end parameter of the `print()` function to avoid two newlines when printing.

Alternatively, we can use the `readline()` method to read individual lines of a file. This method reads a file till the newline, including the newline character.

```
>>> f.readline()
'This is my first file\n'

>>> f.readline()
'This file\n'

>>> f.readline()
'contains three lines\n'

>>> f.readline()
''
```

Lastly, the `readlines()` method returns a list of remaining lines of the entire file. All these reading methods return empty values when the end of file (EOF) is reached.

```
>>> f.readlines()
['This is my first file\n', 'This file\n', 'contains three lines\n']
```

Python File Methods

There are various methods available with the file object. Some of them have been used in the above examples.

Here is the complete list of methods in text mode with a brief description:

Method	Description
close()	Closes an opened file. It has no effect if the file is already closed.
detach()	Separates the underlying binary buffer from the <code>TextIOBase</code> and returns it.
fileno()	Returns an integer number (file descriptor) of the file.
flush()	Flushes the write buffer of the file stream.
isatty()	Returns <code>True</code> if the file stream is interactive.
read(<code>n</code>)	Reads at most <code>n</code> characters from the file. Reads till end of file if it is negative or <code>None</code> .
readable()	Returns <code>True</code> if the file stream can be read from.
readline(<code>n=-1</code>)	Reads and returns one line from the file. Reads in at most <code>n</code> bytes if specified.
readlines(<code>n=-1</code>)	Reads and returns a list of lines from the file. Reads in at most <code>n</code> bytes/characters if specified.

seek(offset, from=SEEK_SET)	Changes the file position to offset bytes, in reference to from (start, current, end).
seekable()	Returns True if the file stream supports random access.
tell()	Returns an integer that represents the current position of the file's object.
truncate(size=None)	Resizes the file stream to size bytes. If size is not specified, resizes to current location.
writable()	Returns True if the file stream can be written to.
write(s)	Writes the string s to the file and returns the number of characters written.
writelines(lines)	Writes a list of lines to the file.

QUESTION:-

1. How do you read a file and write to another file in Python??
- 2 What are I O operations in Python?
- 3 What is the mode of both writing and reading in binary format in file in Python?
4. What are the read & write file operation in Python?
5. What are different types of files in Python?
6. How do you write data to a file in Python?



Yashaswi Education Society's

Reg No. Maha. : 417/2007/Pune

INTERNATIONAL INSTITUTE OF MANAGEMENT SCIENCE

An ISO 9001 Certified Institute

(Approved by AICTE Ministry of HRD Govt. of India, Recognised by Govt. of Maharashtra
and Affiliated to Savitribai Phule Pune University)

Campus. : IIMS Bldg, S. No. 169/1/A, Opp. Elpro International, Chinchwad, Pune - 411033. Ph.: (020) 27353730/32/33/34, Fax : (020) 27354731
Website. : www.iims.ac.in E-mail : info@iims.ac.in

CONCLUSION:- -----
